

实验 6 硬盘读写与外部程序加载（选做）

特别注意：

- 选做实验可选以下**二选一**方式完成
- 1. 完成本实验手册要求的硬盘读写与外部程序加载的实验
- 2. 使用 Rust 实现实验 3 **和**实验 4 中新增的函数，并且成功编译运行（即使使用 Rust 实现的实验 3 或 4 已经在 OBE 系统相应实验条目下提交，仍需按照实验 6 的要求重新提交，包括不限于修改实验报告的名称、说明实验报告对应实验 6 等），另外，实验 5 可用实验 3 或实验 4 之一的 Rust 实现代替，但选做实验 6 需得 3 和 4 两个实验均用 Rust 实现——**使用这种方式提交的，需要与任课教师约时间，现场演示**

截止时间：2026 年 7 月 17 日 23:59

提交要求：

- 正常提交→OBE 上“实验 6”；
- **注意!!!** 超过本实验所声明截止时间之后的提交均不会被纳入最终成绩核算（如果在截止时间之前有过提交，但截止时间之后又做了覆盖提交，使得在测试评分时看到的是截止时间之后的提交，则记作截止时间之后提交，不参与评分——如果截止时间之后确有修改，通过邮件发送 hjj@ruc.edu.cn，并在邮件中说明情况；如果采用方式二，需实际完成实验 6 的提交才会计入实验 6，但可以实验 3 和 4 的提交时间作为本实验的提交时间）

“选做”实验说明：

- 本次实验为**可选做实验**，主要面向想要在本课堂取得最终成绩 90 分以上的同学设置，如果不打算冲击 90 分以上，可以不用做该实验
- 完成该实验的同学，该实验最终占比 10%；没有做本实验的同学本学期最终成绩不超过 89

实验报告编制要求：

- 文件名：**学号姓名.pdf** **←注意**
- 报告标题：体现出“实验 6”等信息 **←注意**

- 报告作者：学号 姓名 ←注意
- 不同题目之间有明显的间隔，要能够容易地区分不同题目对应的实验内容
- 在报告开篇说明新增的文件和修改过的原有代码文件（包括在前几次实验中编写但在此次实验中做出修改的代码）←注意
- 实验报告不做任何其他版式上的要求
- 如支持了实验中未明确要求的功能键（如更好的文件系统支持等），在实验报告中明确说明对应的代码实现的位置、展示自测效果图并添加说明

提交要求：

- 提交完整可编译运行的源代码（提交前在 `compile` 目录执行 `make clean` 清除二进制文件以减少体积）
 - ◆ 所有代码在提交前必须保证能在实验 2 所提及的服务器上编译通过（非 Rust 部分）
- 实验报告与源码目录一起压缩，压缩包必须使用 Linux 常用的压缩包格式 `.tar.gz`，源码不用额外打包，源码目录与实验报告一起打包。
 - ◆ 假设源码目录包 `Xinu-qemu-2026` 和实验报告 `123456 学生.pdf` 在同一个目录
 - ◆ 在该目录中执行：`tar czvf 学号.tar.gz .`
 - ◆ 压缩包文件名为学号，上述命令最后有一个“.”

代码编写要求：

- 函数命名规则
 - ◆ 所有新增的内核态函数均以“`k 学号_`”作为前缀，例：学号为 123456 的同学实现了一个内核态 `printf` 函数，则该函数的函数名应为“`k123456_printf`”
 - ◆ 所有新增的用户态函数均以“`u 学号_`”作为前缀，例：学号为 123456 的同学实现了一个用户态 `printf` 函数，则该函数的函数名应为“`u123456_printf`”
 - ◆ 在本实验手册内提及、以及代码中增加的新函数均应按照上述要求

命名

- ◆ 此处所指的函数包括 C 语言函数和汇编代码函数
- **全局变量、结构体命名规则**
 - ◆ 所有新增的全局变量和结构体的命名也遵循函数命名规则
 - ◆ 所有在原有数据结构中新增的字段，以“**f 学号_**”作为前缀，新增结构体中的字段无需遵循此规则
 - ◆ 局部变量不用遵循上述规则
- 所有新增的其它结构体（如果需要）、全局变量声明（如果需要）、新的函数声明（如果需要），放到 include/**Lab6.h** 中，该头文件可在 xinu.h 中引用 **←注意**
- 所有新增的功能函数，放入 system 目录下以“Lab6_”为前缀名的源文件中；如果所有函数都在同一个源文件中，则该文件名称可为 Lab6.c；也可按照 Xinu 的习惯以函数本身的功能名称参与文件名，如 Lab6_printf.c **←注意**
- **对原有代码的修改：**单行改动，应以修改行前一行或当前行尾的注释“// **Lab6 学号**”标记；多行修改，应以首行行前“/***Lab6 学号: Begin***/”和末行行后“/***Lab6 学号: End***/”标记
- 在 shell 目录下新建一个文件 xsh_lab6.c，其中包含两个 shell 命令函数“xsh_lab6_ls”和“xsh_lab6_run”，参数类型请参考其他 shell 命令，详情参考本实验手册 2.3 和 2.4
- 硬盘驱动程序实现到 device 目录中自命名的目录中，文件系统实现可以直接放在 system 目录下，也能另外添加目录

辅助代码及说明：

- 在 OBE “课件”中有一个 fs_util.zip 压缩包，其中包含了创建示例硬盘镜像并将多个文件按照一定文件系统格式放入硬盘镜像的程序 mkfs.c，并附上了一个示例程序 test 所需的源文件
- 压缩包还包含一个 **elf.h**，可添加到 Xinu 头文件中，用于读取 ELF 可执行程序
- **在 Xinu 源代码目录下创建 fs_util 目录**，并将压缩包中所有文件放入该

目录下；在提交代码时，该目录一起提交，但提交前请删除生成的 fs.img 和.o 文件等

- 根据后续实验功能添加自己的测试功能代码到该目录，并修改 Makefile，以便能在该目录下执行 make 时将所需的文件放入硬盘镜像（硬盘镜像默认名称为 fs.img）
- 放入硬盘镜像中的文件，除了后续实验所要求的可执行程序之外，还应该至少包含一个纯文本文件（如某个源码文件）
- 示例硬盘镜像类的文件系统、如何存放文件到硬盘镜像、二进制 ELF 可执行程序的基本格式等信息，可参考 README 文件
- 最后评分测试时，可能会将其他的文件和程序放入硬盘镜像参与测试
- 如果自己实现了支持更好的文件系统的内核，并采用其他文件系统来格式化硬盘，请附上相应的创建硬盘镜像、格式化硬盘镜像、装入文件等功能的源代码和 Makefile，并在实验报告中说明如何使用
- 假设使用 fs_util/fs.img 作为硬盘镜像，在 Xinu 的 compile 目录下使用以下命令可以将该硬盘载入 QEMU，硬盘为主通道下的从盘（注意：index=1）。请在实验报告靠前的位置明确写明正确运行所需的命令（以 compile 目录为当前路径，比如使用了不同的 img 路径、index 编号等，以方便测试）

```
qemu-system-i386 -kernel xinu.elf \
    -drive file=../fs_util/fs.img,index=1,media=disk,format=raw
```

本次实验要求提交代码（参看前文“提交要求”），也需提交实验报告。实验报告应说明对代码的修改（新增、修改等），以及丰富的截图和配套的文字说明（不能只有截图而无文字说明，或只有文字而无截图）。

1. 实验目的

希望学生能掌握硬盘驱动程序开发和外部程序装载，将 Xinu 转化为一个初步支持硬盘设备、并支持从硬盘动态加载程序到内存执行的操作系统内核。

2. 实验要求

【必做：2.1, 2.2, 2.3, 2.4, 2.5】

2.1 实现硬盘驱动程序及简易文件系统操作，支持从硬盘动态加载程序

- a 要求本实验与实验 5（键盘显示器驱动程序）存在依赖关系，即本实验完成的命令需要在实验 5 环境下可执行（不依赖原有 tty 驱动完成键盘输入、显示器输出）

2.2 以附带的硬盘镜像和简易文件系统为例（实现其他文件系统应完成相似要求），应完成如下要求：

- a 支持通过 Xinu 抽象 IO 接口调用相应的驱动程序
- b 至少支持通过 read 和 write 两个接口读取硬盘数据和向硬盘写入数据，其它接口根据实际需求实现
- c read 和 write 对应的驱动函数的参数分别表示何种语义，应在代码注释和实验报告中说明，可参考 device/ram 中相关函数
- d 支持通过硬盘中断处理的方式来读写硬盘数据（如果 QEMU 支持，也可采用 DMA 方式——自行查阅资料并实现，在实验报告中说明）
- e 支持硬盘读写保护，即任意时刻只有一个进程在读或写硬盘（每次操作前注意检查硬盘是否适合进行读写）
- f 硬盘驱动程序应放入 device 目录下相应的目录，并修改 Xinu 的 Makefile，确保驱动程序被正确编译集成到 xinu.elf 中

2.3 实现至少 2 个用户态测试程序，名称自拟，以下以 A 和 B 称呼，应满足如下要求：

- a 两个程序均应能够运行在用户态，需要内核态操作时（如输入输出），通过系统调用与内核交互
- b A 和 B 的基本形式可参考 fs_util 中的 test，编译命令可参考相应的 Makefile 和 README，程序的入口点应该在虚拟内存空间中的某个位置，程序自身的代码段、数据段等所有来自于程序二进制 ELF 文件的内容都应该处于用户态的虚拟内存空间中（fs_util 中示例代码的代码段入

口地址为 0x80001000，可修改目录下的 Makefile，将入口地址放到合适的地方——根据自己实现虚拟内存机制设置入口地址即可，在实验报告中给出说明）

- c 程序 A 应实现类似如下的功能，即，①输出局部变量的地址和 main 函数的地址，②以字符串形式，将传入的参数输出到屏幕（本实验中，所有提及的屏幕，都是实验 5 完成的 VGA 屏幕，但也可以同时输出到以前的 tty 串口）——**注意：**与普通桌面应用程序保持一致，程序的第 0 个参数是程序本身的名字，③输出学号和姓名（输出中文姓名如果显示错误，可使用拼音替代）

```
// 请自行保证代码正确性
int32 x;
printf("&x=0x%x, &main=0x%x\n", &x, &main);
for (int i = 0; i < argc; i++) {
    printf("arg-%d: %s\n", i, argv[i]);
}
printf("<学号> <姓名>\n");
```

- d 程序 B 接收一个参数，该参数是硬盘镜像中已经存在的某个文件的文件名，程序 B 将硬编码的一小段自定义字符串写入指定文件的起始位置（程序 B 中可以不用考虑指定文件名错误、指定文件不存在等错误情况），写入之后需检查硬盘镜像中确实已经写入
- e 程序 A 和 B 中的屏幕输出、读写硬盘都应该通过系统调用与内核相应功能进行交互（**注意：**A 和 B 的二进制代码存在于 fs.img 硬盘镜像中，没有编译集成进 xinu.elf 中）
- f 程序 A 和 B 的源代码放入 fs_util 中，修改 Makefile，使得可以通过 make 命令编译生成二进制可执行程序并存放到 fs.img 中

2.4 实现 shell 命令程序 xsh_lab6_ls，使其完成如下要求：

- a 对应 shell 命令 ls
- b 列出硬盘镜像中所有的文件，至少列出文件名，以便其他命令或上述的程序 B 可使用，可列出每个文件的大小和硬盘内偏移（参考 fs_util/README 相关信息）

2.5 实现 shell 命令程序 xsh_lab6_run，使其完成如下要求：

- a 对应 shell 命令 run

- b 该命令接收至少一个参数，如“run A”，第 1 个参数为待执行的用户态程序，该程序来自于硬盘镜像，其名称可从上一步的 lab6_ls 命令的输出中获得——如果实现了其他文件系统（如含目录），外部程序可以完整路径的方式作为参数，如“run /dir/A”
- c 如果目标程序需要参数，如上述的程序 A，则 A 的参数紧接 A，以空格分割，如“run A hello world”
- d 该 shell 命令应根据参数信息，从硬盘镜像中载入相应的二进制程序，创建对应的进程，并传入相应的参数，后续通过进程调度使目标进程能在用户态正常执行
- e 读写硬盘应通过抽象 IO 接口进行，而不是直接调用硬盘驱动程序
- f run 命令应通过用户态系统调用 fork 和 exec 完成外部命令的加载和运行，需要修改实验 4 所实现的 fork 和 exec。如果有修改，在实验报告中说明
- g （扩展要求，不强制）如果指定的目标程序文件名错误或者不是可执行 ELF 文件，输出错误提示信息并退出——建议至少在实验报告中列出自己提交的代码编译生成后哪些名称是可作为普通文本文件、哪些是可作为外部程序

3. 思维扩展

（不要求在实验报告中作答）

- 3.1 添加有效的文件系统支持，进程或命令读写文件不用直接与硬盘设备交互（即使是通过抽象接口），而是通过文件系统接口进行交互
- 3.2 查看并了解 OpenHarmony 中的块设备驱动及文件系统实现。